

数字设计与计算机原理复习要点

第五章：DDCA Chapter 3 – 时序逻辑设计

1 时序逻辑导论 (pp.3–4)

- 时序逻辑输出取决于当前和之前的输入——具有记忆 [p.3]
- 状态 (State): 描述电路未来行为所需的所有信息
- 锁存器和触发器 (Latch & Flip-flop): 存储 1 位状态的状态元件
- 同步时序电路: 组合逻辑 + 一组触发器 [p.3]

2 状态元件 (pp.5–30)

双稳态电路 (Bistable Circuit) [pp.6–7]

两个反相器交叉耦合，两个输出： Q 和 $\overline{Q}$ ，无输入，是其他状态元件的基础。

SR 锁存器 (SR Latch) [pp.8–15]

- S (Set) = 1: 置位, $Q = 1$
- R (Reset) = 1: 复位, $Q = 0$
- $S=R=0$: 保持状态
- $S=R=1$: 禁止 (非法输入, $Q = \overline{Q} = 0$ 产生竞争)

D 锁存器 (D Latch) [pp.16–22]

- 数据 (D) 输入 + 使能 (CLK/EN) 输入
- $CLK=1$ 时透明 ($Q = D$), $CLK=0$ 时保持
- 消除了 SR 锁存器的非法输入问题

D 触发器 (D Flip-flop) [pp.23–30]

- 边沿触发 (Edge-triggered): 仅在时钟上升沿 (或下降沿) 采样 D
- 通常由两个 D 锁存器级联构成 (主从结构)
- 输出在时钟边沿之后更新

3 同步逻辑设计 (pp.31–50)

- 所有状态元件使用同一个时钟
- 组合逻辑位于触发器之间
- 设计流程: 状态转移图 $\rightarrow$ 状态转移表 $\rightarrow$ 次态逻辑 $\rightarrow$ 电路实现

- **建立时间 (Setup Time) t_{setup} :** 时钟边沿前数据必须稳定的最短时间
- **保持时间 (Hold Time) t_{hold} :** 时钟边沿后数据必须保持的最短时间
- **时钟到 Q 延迟 (Clock-to-Q Delay) t_{pcq} :** 时钟边沿到输出更新的延迟

4 有限状态机 FSM (Finite State Machines) (pp.51–85)

Moore 型 vs Mealy 型

- **Moore FSM:** 输出仅取决于当前状态
- **Mealy FSM:** 输出取决于当前状态和当前输入

FSM 设计步骤

1. 确定输入/输出
2. 画状态转移图 (State Transition Diagram)
3. 建立状态转移表 (State Transition Table)
4. 选择状态编码 (二进制编码、独热码等)
5. 推导次态逻辑和输出逻辑方程
6. 画电路原理图

状态编码

- **二进制编码:** 最少触发器数 ($\lceil \log_2 N \rceil$ 个用于 N 个状态)
- **独热码 (One-Hot):** 每个状态一个触发器, N 个状态需 N 个触发器。速度快, 但面积大

5 时序逻辑的时序 (pp.86–100)

时序约束

- **最小时钟周期:** $T_c \geq t_{pcq} + t_{pd} + t_{setup}$
- t_{pcq} : 时钟到 Q 的传播延迟
- t_{pd} : 组合逻辑传播延迟
- t_{setup} : 触发器建立时间
- **保持时间约束:** $t_{cd} \geq t_{hold} - t_{ccq}$
- t_{cd} : 组合逻辑污染延迟
- **亚稳态 (Metastability):** 当输入在建立/保持窗口内变化时, 输出可能进入不确定的中间状态

6 并行性 (Parallelism) (pp.101–111)

- **空间并行:** 多个相同硬件单元同时处理不同数据
- **时间并行 (流水线):** Pipelining – 将组合逻辑分成多个阶段, 中间插入触发器
- **流水线优势:** 提高吞吐量 (Throughput), 代价是增加延迟 (Latency)
- **流水线级数:** 将长关键路径切成多段, 提高时钟频率

7 易错点与技巧

- 易错点 1: 混淆锁存器（电平敏感）和触发器（边沿敏感）
- 易错点 2: 忘记处理 FSM 中未使用的状态
- 易错点 3: 建立/保持时间违规导致亚稳态
- 技巧: FSM 设计时先用 Digital 或状态图可视化，再写 HDL 代码
- 技巧: 用独热码编码 FSM 可简化输出逻辑